Part A

Throughout the development of Pixel Coliseum, I was solely responsible for the design, implementation, and iteration of the entire system. I built both the frontend and backend, using a Phaser-based client and a Node.js/Colyseus server to support real-time multiplayer gameplay. I implemented core systems such as player movement, client-side prediction, server-authoritative state synchronization, combat mechanics (melee and projectile), and basic enemy AI. I also integrate a PostgreSQL database to manage user data and designed the overall game flow, including authentication, lobby management, and gameplay phases. This required me to independently make architectural decisions, debug complex interactions between systems, and ensure the game functioned smoothly across multiple clients.

This project directly built upon the skills I identified in my initial assessment, particularly in full-stack development, real-time systems, and problem-solving. I strengthened my understanding of networking concepts like latency handling and synchronization, and gained hands-on experience debugging issues such as desync, collision handling, and coordinate mismatches from map data. One of my biggest successes was achieving responsive and consistent multiplayer gameplay using client-side prediction and reconciliation. However, I also faced challenges, especially when diagnosing subtle bugs across the client-server boundary and managing the complexity of a growing codebase without a team. Through these obstacles, I developed stronger debugging strategies, improved my ability to break down complex problems, and gained confidence in building complete systems independently.

Part B

Although this was a solo project, I approached it with the mindset of a full team by taking on multiple roles, including frontend developer, backend developer, and system designer. As a “team of one,” I successfully delivered a functional multiplayer game that demonstrates core software engineering principles such as modular design, real-time communication, and scalable architecture. I learned that even without multiple contributors, clear organization, planning, and iterative development are essential to maintaining progress. Treating different components of the project as if they were owned by separate team members helped me stay structured and disciplined in my approach.

Working independently also highlighted both the strengths and limitations of solo development. On one hand, I benefited from full control over decisions, allowing for fast

iteration and a consistent vision. On the other hand, I did not have the advantage of peer feedback, code reviews, or shared problem-solving, which made debugging and design validation more challenging. This experience reinforced the importance of collaboration and communication in team environments. Since I was the only contributor, all progress and outcomes were the result of my own efforts, and there are no teammates to compare against or single out for recognition. However, this project demonstrates my ability to take initiative, remain self-motivated, and deliver a complete, complex system independently.